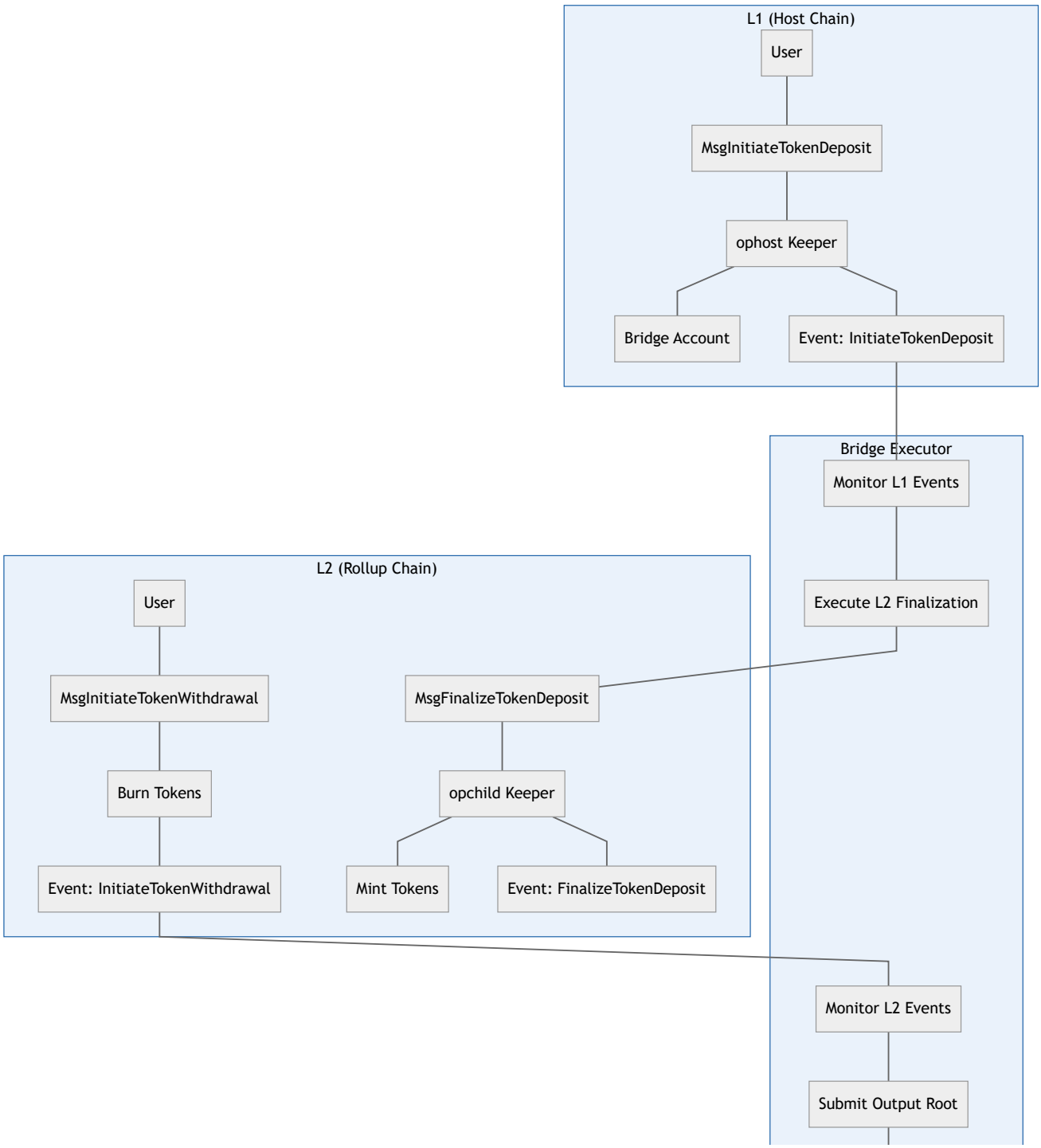
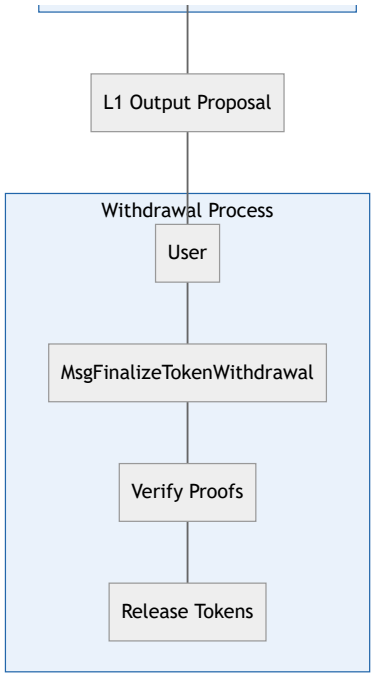
04. 브리지 메커니즘 동작 원리 분석

개요

OPinit의 브리지 메커니즘은 L1(호스트 체인)과 L2(롤업 체인) 간의 토큰 전송을 안전하고 효율적으로 처리하는 핵심 시스템입니다. 이 문서는 입금(L1→L2)과 출금(L2→L1) 프로세스의 상세한 동작 원리와 BridgeExecutor의 역할을 실제 코드 구현을 바탕으로 분석합니다.

브리지 아키텍처 개요


L1 (Host Chain)
User
MsgInitiateTokenDeposit
ophost Keeper
Bridge Account
Event: InitiateTokenDeposit
Bridge Executor
Monitor L1 Events
Execute L2 Finalization
L2 (Rollup Chain)
User
MsgInitiateTokenWithdrawal
MsgFinalizeTokenDeposit
Burn Tokens
opchild Keeper
Event: InitiateTokenWithdrawal
Mint Tokens
Event: FinalizeTokenDeposit
Monitor L2 Events
Submit Output Root



L1에서 L2로의 입금 프로세스

1. 입금 시작 (MsgInitiateTokenDeposit)

사용자가 L1에서 토큰을 L2로 전송하려면 `MsgInitiateTokenDeposit` 메시지를 실행합니다.

메시지 구조

```
type MsgInitiateTokenDeposit struct {
Sender  string  // L1 송신자 주소
```

```
To  string  // L2 수신자 주소
Amount sdk.Coin // 전송할 토큰 수량
BridgeId  uint64  // 브리지 ID
Data []byte  // 추가 데이터 (훅 실행용)
}
```

L1 처리 과정 (ophost/keeper/msg_server.go)

```
func (ms MsgServer) InitiateTokenDeposit(ctx context.Context, req *types.MsgInit
// 1. 메시지 검증
if  err := req.Validate(ms.authKeeper.AddressCodec());  err != nil {
return  nil, err
}

// 2. 브리지 존재 확인
if  ok, err := ms.BridgeConfigs.Has(ctx, bridgeId);  err != nil {
return  nil, err
```

```
} else  if !ok {
return  nil, types.ErrBridgeNotFound
}

// 3. L1 시퀀스 번호 증가
l1Sequence, err := ms.IncreaseNextL1Sequence(ctx, bridgeId)
if  err != nil {
return  nil, err
}

// 4. 토큰을 브리지 계정으로 전송 (양수인 경우만)
if  coin.IsPositive() {
bridgeAddr := types.BridgeAddress(bridgeId)
if  err := ms.bankKeeper.SendCoins(ctx, sender, bridgeAddr, sdk.NewCoins(coin));
return  nil, err
}
}

// 5. 토큰 페어 등록
l2Denom := types.L2Denom(bridgeId, coin.Denom)
if  ok, err := ms.HasTokenPair(ctx, bridgeId, l2Denom);  err != nil {
return  nil, err
} else  if !ok {
```

```go
if  err := ms.SetTokenPair(ctx, bridgeId, l2Denom, coin.Denom);  err != nil {
return  nil, err
}
}

// 6. 이벤트 발생 (BridgeExecutor가 모니터링)
sdk.UnwrapSDKContext(ctx).EventManager().EmitEvent(sdk.NewEvent(
types.EventTypeInitiateTokenDeposit,
sdk.NewAttribute(types.AttributeKeyBridgeId, strconv.FormatUint(bridgeId, 10)),
sdk.NewAttribute(types.AttributeKeyL1Sequence, strconv.FormatUint(l1Sequence, 10
sdk.NewAttribute(types.AttributeKeyFrom, req.Sender),
sdk.NewAttribute(types.AttributeKeyTo, req.To),
sdk.NewAttribute(types.AttributeKeyL1Denom, coin.Denom),
sdk.NewAttribute(types.AttributeKeyL2Denom, l2Denom),
sdk.NewAttribute(types.AttributeKeyAmount, coin.Amount.String()),
sdk.NewAttribute(types.AttributeKeyData, hex.EncodeToString(req.Data)),
))

return &types.MsgInitiateTokenDepositResponse{
Sequence: l1Sequence,
}, nil
}
```

2. BridgeExecutor의 역할

BridgeExecutor는 L1과 L2 간의 메시지 중계를 담당하는 오프체인 컴포넌트입니다:

1. **L1 이벤트 모니터링**: `EventTypeInitiateTokenDeposit` 이벤트를 감지
2. **L2 트랜잭션 실행**: 감지된 입금을 L2에서 완료
3. **L2 이벤트 모니터링**: `EventTypeInitiateTokenWithdrawal` 이벤트를 감지
4. **Output Root 제출**: L2 상태를 L1에 제출

3. 입금 완료 (MsgFinalizeTokenDeposit)

BridgeExecutor가 L1 입금 이벤트를 감지하면 L2에서 `MsgFinalizeTokenDeposit` 을 실행합니다.

메시지 구조

```go
type MsgFinalizeTokenDeposit struct {
Sender  string  // BridgeExecutor 주소
From  string  // L1 송신자 주소
To  string  // L2 수신자 주소
Amount sdk.Coin // 토큰 수량
Sequence  uint64  // L1 시퀀스 번호
Height  uint64  // L1 블록 높이
BaseDenom  string  // 기본 토큰명
Data []byte  // 추가 데이터
}
```

L2 처리 과정 (opchild/keeper/msg_server.go)

```go
func (ms MsgServer) FinalizeTokenDeposit(ctx context.Context, req *types.MsgFina

// 1. 권한 확인 (BridgeExecutor만 실행 가능)

if err := ms.checkBridgeExecutorPermission(ctx, req.Sender); err != nil {

return nil, err

}


// 2. 시퀀스 검증 (순차 처리 보장)

finalizedL1Sequence, err := ms.GetNextL1Sequence(ctx)

if err != nil {

return nil, err

}


if req.Sequence < finalizedL1Sequence {

// 이미 처리된 경우 No-op

return &types.MsgFinalizeTokenDepositResponse{Result: types.NOOP}, nil

} else if req.Sequence > finalizedL1Sequence {

return nil, types.ErrInvalidSequence

}


// 3. 토큰 입금 처리

var depositSuccess bool

var reason string
```

```
toAddr, err := ms.authKeeper.AddressCodec().StringToBytes(req.To)
if err != nil {
depositSuccess = false
reason = fmt.Sprintf("failed to convert recipient address: %s", err)
} else {
depositSuccess, reason = ms.safeDepositToken(ctx, toAddr, sdk.NewCoins(coin))
}

// 4. L1 시퀀스 업데이트
if _, err := ms.IncreaseNextL1Sequence(ctx); err != nil {
return nil, err
}

// 5. 토큰 메타데이터 등록
if ok := ms.bankKeeper.HasDenomMetaData(ctx, coin.Denom); !ok {
ms.setDenomMetadata(ctx, req.BaseDenom, coin.Denom)
}

// 6. 토큰 페어 등록 및 토큰 생성
if ok, err := ms.DenomPairs.Has(ctx, coin.Denom); err != nil {
return nil, err
} else if !ok {
if err := ms.DenomPairs.Set(ctx, coin.Denom, req.BaseDenom); err != nil {
```

```
return nil, err
}

// 토큰이 존재하지 않으면 생성
if ms.tokenCreationFn != nil {
if err := ms.tokenCreationFn(ctx, coin.Denom, 0); err != nil {
return nil, err
}
}
}

// 7. 입금 실패 시 자동 환불 처리
if !depositSuccess && coin.IsPositive() {
l2Sequence, err := ms.IncreaseNextL2Sequence(ctx)
if err != nil {
return nil, err
}

err = ms.emitWithdrawEvents(ctx, types.NewMsgInitiateTokenWithdrawal(req.To, req
if err != nil {
return nil, err
}
}
```

```
return &types.MsgFinalizeTokenDepositResponse{Result: types.SUCCESS}, nil
}
```

L2에서 L1으로의 출금 프로세스

1. 출금 시작 (MsgInitiateTokenWithdrawal)

사용자가 L2에서 토큰을 L1으로 전송하려면 `MsgInitiateTokenWithdrawal` 메시지를 실행합니다.

메시지 구조

```
type MsgInitiateTokenWithdrawal struct {
Sender  string  // L2 송신자 주소
To  string  // L1 수신자 주소
Amount sdk.Coin // 출금할 토큰 수량
}
```

L2 처리 과정 (opchild/keeper/msg_server.go)

```
func (ms MsgServer) InitiateTokenWithdrawal(ctx context.Context, req *types.MsgI
// 1. 메시지 검증
if  err := req.Validate(ms.authKeeper.AddressCodec());  err != nil {
return  nil, err
}
```

```
coin := req.Amount

burnCoins := sdk.NewCoins(coin)


senderAddr, err := ms.authKeeper.AddressCodec().StringToBytes(req.Sender)

if  err != nil {

return  nil, err

}


// 2. 마이그레이션된 토큰 처리 확인

if  handled, err := ms.Keeper.HandleMigratedTokenWithdrawal(ctx, req);  err != n

return  nil, err

} else  if  handled {

return &types.MsgInitiateTokenWithdrawalResponse{}, nil

}


// 3. 토큰을 모듈 계정으로 전송

if  err := ms.bankKeeper.SendCoinsFromAccountToModule(ctx, senderAddr, types.Mod

return  nil, err

}


// 4. 토큰 소각

if  err := ms.bankKeeper.BurnCoins(ctx, types.ModuleName, burnCoins);  err != ni
```

```go
return  nil, err
}

// 5. L2 시퀀스 번호 증가
l2Sequence, err := ms.IncreaseNextL2Sequence(ctx)
if  err != nil {
return  nil, err
}

// 6. 출금 이벤트 발생
err = ms.emitWithdrawEvents(ctx, req, l2Sequence)
if  err != nil {
return  nil, err
}

return &types.MsgInitiateTokenWithdrawalResponse{
Sequence: l2Sequence,
}, nil
}
```

2. Output Root 제출

BridgeExecutor는 L2의 출금 이벤트를 모니터링하고 주기적으로 L2 상태를 L1에 제출합니다.

MsgProposeOutput 처리 (ophost/keeper/msg_server.go)

```go
func (ms MsgServer) ProposeOutput(ctx context.Context, req *types.MsgProposeOutp

// 1. 권한 확인 (Proposer만 실행 가능)

if  proposer != bridgeConfig.Proposer {

return  nil, sdkerrors.ErrUnauthorized.Wrap("invalid proposer")

}



// 2. Output Index 증가

outputIndex, err := ms.IncreaseNextOutputIndex(ctx, bridgeId)

if  err != nil {

return  nil, err

}



// 3. 블록 번호 검증 (순차 증가 확인)

if  outputIndex != 1 {

lastOutputProposal, err := ms.GetOutputProposal(ctx, bridgeId, outputIndex-1)

if  err != nil {

return  nil, err

}



if  l2BlockNumber <= lastOutputProposal.L2BlockNumber {

return  nil, types.ErrInvalidL2BlockNumber.Wrapf("last %d, got %d", lastOutputPr
```

```
}
}

// 4. Output Proposal 저장
if err := ms.SetOutputProposal(ctx, bridgeId, outputIndex, types.Output{
OutputRoot: outputRoot,
L1BlockNumber: uint64(sdkCtx.BlockHeight()),
L1BlockTime: sdkCtx.BlockTime(),
L2BlockNumber: l2BlockNumber,
}); err != nil {
return nil, err
}

return &types.MsgProposeOutputResponse{}, nil
}
```

3. 출금 완료 (MsgFinalizeTokenWithdrawal)

Finalization Period가 지난 후 사용자는 Merkle Proof를 제공하여 출금을 완료할 수 있습니다.

메시지 구조

```
type MsgFinalizeTokenWithdrawal struct {
Sender string // 실행자 주소
BridgeId uint64 // 브리지 ID
```

```
OutputIndex uint64 // Output Index
WithdrawalProofs [][]byte // Merkle Proof
From string // L2 송신자 주소
To string // L1 수신자 주소
Sequence uint64 // L2 시퀀스 번호
Amount sdk.Coin // 토큰 수량
Version []byte // 버전 정보
StorageRoot []byte // Storage Root
LastBlockHash []byte // 마지막 블록 해시
}
```

L1 처리 과정 (ophost/keeper/msg_server.go)

```
func (ms MsgServer) FinalizeTokenWithdrawal(ctx context.Context, req *types.MsgF
// 1. Finalization 확인
if ok, err := ms.IsFinalized(ctx, bridgeId, outputIndex); err != nil {
return nil, err
} else if !ok {
return nil, types.ErrNotFinalized
}

// 2. Output Root 검증
outputProposal, err := ms.GetOutputProposal(ctx, bridgeId, outputIndex)
```

```go
if  err != nil {
return  nil, err
}

outputRoot := types.GenerateOutputRoot(req.Version[0], req.StorageRoot, req.Last
if !bytes.Equal(outputProposal.OutputRoot, outputRoot[:]) {
return  nil, types.ErrFailedToVerifyWithdrawal.Wrap("invalid output root")
}

// 3. 중복 출금 방지 확인
withdrawalHash := types.GenerateWithdrawalHash(bridgeId, l2Sequence, req.From, r
if  ok, err := ms.HasProvenWithdrawal(ctx, bridgeId, withdrawalHash);  err != ni
return  nil, err
} else  if  ok {
return  nil, types.ErrWithdrawalAlreadyFinalized
}

// 4. Merkle Proof 검증
rootHash := types.GenerateRootHashFromProofs(withdrawalHash, req.WithdrawalProof
if !bytes.Equal(req.StorageRoot, rootHash[:]) {
return  nil, types.ErrFailedToVerifyWithdrawal.Wrap("invalid storage root proofs
}
```

```
// 5. 출금 기록 저장
if err := ms.RecordProvenWithdrawal(ctx, bridgeId, withdrawalHash); err != nil
return nil, err
}

// 6. 토큰 전송 (마이그레이션되지 않은 경우)
if handled, err := ms.Keeper.HandleMigratedTokenWithdrawal(ctx, req); err != n
return nil, err
} else if handled {
return &types.MsgFinalizeTokenWithdrawalResponse{}, nil
} else {
receiver, err := ms.authKeeper.AddressCodec().StringToBytes(req.To)
if err != nil {
return nil, err
}

bridgeAddr := types.BridgeAddress(bridgeId)
if err := ms.bankKeeper.SendCoins(ctx, bridgeAddr, receiver, sdk.NewCoins(sdk.N
return nil, err
}
}

return &types.MsgFinalizeTokenWithdrawalResponse{}, nil
```

```
}
```

Withdrawal Proving 메커니즘

1. Withdrawal Hash 생성

각 출금 트랜잭션은 고유한 해시로 식별됩니다:

```
func GenerateWithdrawalHash(bridgeId uint64, sequence uint64, from, to, denom

var data []byte

data = append(data, sdk.Uint64ToBigEndian(bridgeId)...)

data = append(data, sdk.Uint64ToBigEndian(sequence)...)

data = append(data, []byte(from)...)

data = append(data, []byte(to)...)

data = append(data, []byte(denom)...)

data = append(data, sdk.Uint64ToBigEndian(amount)...)

hash := sha3.Sum256(data)

return hash[:]

}
```

2. Merkle Tree 구성

BridgeExecutor는 주기적으로 모든 출금 트랜잭션을 포함하는 정렬된 Merkle Tree를 구성합니다:

```
func GenerateRootHashFromProofs(leafHash []byte, proofs [][]byte) [32]byte {
```

```
currentHash := leafHash
for _, proof := range proofs {
if bytes.Compare(currentHash, proof) < 0 {
// currentHash < proof
combined := append(currentHash, proof...)
hash := sha3.Sum256(combined)
currentHash = hash[:]
} else {
// currentHash >= proof
combined := append(proof, currentHash...)
hash := sha3.Sum256(combined)
currentHash = hash[:]
}
}
var result [32]byte
copy(result[:], currentHash)
return result
}
```

3. Output Root 생성

L2의 상태는 다음과 같이 Output Root로 요약됩니다:

```
func GenerateOutputRoot(version byte, storageRoot, lastBlockHash []byte) [32]b
```

```go
var data []byte
data = append(data, version)
data = append(data, storageRoot...)
data = append(data, lastBlockHash...)
return sha3.Sum256(data)
}
```

트랜잭션 플로우 요약

입금 플로우 (L1 → L2)

1. **사용자**: `MsgInitiateTokenDeposit` 실행 → L1에서 토큰 락업
2. **BridgeExecutor**: L1 이벤트 감지 → L2에서 `MsgFinalizeTokenDeposit` 실행
3. **L2**: 토큰 민팅 및 사용자에게 전송

출금 플로우 (L2 → L1)

1. **사용자**: `MsgInitiateTokenWithdrawal` 실행 → L2에서 토큰 소각
2. **BridgeExecutor**: L2 이벤트 감지 → Merkle Tree 구성 → `MsgProposeOutput` 실행
3. **Finalization Period 대기** (보통 7일)
4. **사용자**: Merkle Proof와 함께 `MsgFinalizeTokenWithdrawal` 실행 → L1에서 토큰 수령

보안 고려사항

1. 권한 관리

- **BridgeExecutor**: L2에서 입금 완료 및 Oracle 업데이트 권한
- **Proposer**: L1에서 Output Root 제출 권한
- **Challenger**: 잘못된 Output Root 삭제 권한

2. 시퀀스 관리

- L1과 L2 각각 독립적인 시퀀스 번호 관리
- 순차 처리를 통한 중복 실행 방지
- 시퀀스 불일치 시 트랜잭션 거부

3. Finalization Period

- Output Root 제출 후 일정 기간 대기
- Challenger의 이의제기 기간 보장
- 악의적인 Output Root에 대한 방어 메커니즘

4. Merkle Proof 검증

- 정렬된 Merkle Tree를 통한 효율적인 검증
- 중복 출금 방지를 위한 해시 기록
- 암호학적으로 안전한 SHA3-256 해시 사용

이러한 메커니즘을 통해 OPinit은 L1과 L2 간의 안전하고 효율적인 토큰 브리지를 제공합니다.